



MIDDLE - CONNECTOR AS A SERVICE

MCAAS - DES

Database Extractor n Sender

Manual de usuario, configuración, pruebas, build e
instalación

Versión del proyecto: 1.1.0

Plataforma objetivo: Windows x64

Fecha del manual: 15 de septiembre de 2026

Contenido

1. Objetivo y alcance
2. Arquitectura y flujo
3. Funciones soportadas
4. Requisitos pre-lanzamiento
5. Estructura del proyecto
6. Configuración del .env
7. Script SQL dinámico
8. Detección de nuevos y actualizados
9. Modo HISTORICO
10. Errores, reinicio y estado persistente
11. Logs y monitor
12. Gmail API
13. Pruebas y validación
14. Build y empaquetado
15. Instalación en servidor Windows
16. Operación diaria y recuperación
17. Seguridad recomendada
18. Límites y supuestos
19. Referencia rápida de comandos

Idea central: la base origen nunca necesita conocer los destinos. MCAAS - DES lee mediante un SELECT, identifica cambios localmente y replica cada cambio hacia los destinos configurados de forma independiente.

1. Objetivo y alcance

MCAAS - DES unifica extracción, detección de cambios y envío. Desde la versión 1.1.0 una sola instalación puede ejecutar **varios trabajos de extracción**, cada uno con su propia base origen, script SQL, nombre lógico y tabla destino.

Esto permite, por ejemplo, leer empleados desde varias empresas o sucursales de CONTPAQi, o combinar consultas de SQL Server, PostgreSQL y MySQL dentro del mismo servicio. Todos los cambios detectados se intentan reenviar a todos los destinos configurados.

N

extracciones configurables

1-10

destinos por instalación

24/7

operación prevista

Para distinguir datos con claves de negocio repetidas, MCAAS agrega automáticamente un campo de procedencia definido por `ORIGIN_FIELD_NAME`. Su valor es el `EXTRACT_N_NAME` del trabajo que produjo la fila.

Ejemplo: empleado_id=25 de Empleados_Matriz y empleado_id=25 de Empleados_SucursalNorte son entidades distintas porque la identidad funcional incluye también el origen.

2. Arquitectura y flujo

```
EXTRACT_1 -> BD / SQL A --+
EXTRACT_2 -> BD / SQL B --+--> SELECT -> validación -> agrega origen -> hash/estado
EXTRACT_3 -> BD / SQL C --+ |
... +--> Destino 1 -> espera 300 ms
+--> Destino 2 -> espera 300 ms
+--> ... hasta Destino 10
```

Identidad funcional

PK destino autoincremental: id = 1841 (administrada por la BD)

Identidad MCAAS: source_name = "Empleados_Matriz" + empleado_id = 25
Identidad MCAAS: source_name = "Empleados_Sucursal_Norte" + empleado_id = 25

La PK autoincremental del destino no se utiliza para localizar la fila a sincronizar. En modo normal, el `UPDATE` y la comprobación previa al `INSERT` usan la combinación **origen + columnas clave del script**.

Componentes

Componente	Responsabilidad
Extraction jobs	Unidades <code>EXTRACT_N</code> . Cada una encapsula nombre lógico, conexión origen y script SQL.
Extractor	Ejecuta cada SQL de lectura y obtiene columnas/filas sin requerir recompilación.
Origin tagger	Agrega a cada fila la columna configurada por <code>ORIGIN_FIELD_NAME</code> con valor <code>EXTRACT_N_NAME</code> .
Detector de cambios	Genera huellas SHA-256 de la identidad y del contenido completo de la fila.
Estado local	Guarda, por stream y destino, la última huella enviada. Un fallo en un destino no marca como enviado a los demás.
Sender	Construye operaciones parametrizadas para SQL Server, PostgreSQL o MySQL.
Failure tracker	Cuenta fallos dentro de una ventana de tiempo y activa la pausa persistente.
Monitor / Task Scheduler	Mantiene operación 24/7 y una consola de lectura desacoplada del núcleo.

3. Funciones soportadas

- Múltiples extracciones `EXTRACT_1`, `EXTRACT_2`, ... definidas en `.env`.
- Cada extracción puede usar una BD distinta o compartir BD con otras consultas.
- SQL Server, PostgreSQL y MySQL como origen o destino.
- Nombre lógico único por extracción, persistido automáticamente en cada fila destino.
- Identidad por `origen + MCAAS_KEY_COLUMNS`.
- PK autoincremental propia en destino, protegida por `DESTINATION_AUTO_ID_COLUMN`.
- Scripts SQL reemplazables sin recompilar.

- Entre 1 y 10 destinos, cada uno con credenciales y política histórica independientes.
- Modo normal: UPDATE por identidad funcional; si no existe, INSERT.
- Modo `HISTORICO=true`: cada versión nueva/modificada se inserta como nueva fila.
- Pausa configurable de 300 ms entre intento de un destino y el siguiente.
- Estado independiente por extracción y destino.
- Logs operativos, errores y banderas sin guardar los valores de las filas.
- Detección de error persistente y alerta por Gmail API.
- Build Windows autocontenido para servidores sin Node.js.

No crea tablas automáticamente. Las tablas destino deben existir y ser compatibles con las columnas de cada SELECT, más la columna de procedencia.

4. Requisitos pre-lanzamiento

4.1 Bases origen

- [] Cada servidor origen es accesible desde la PC donde correrá MCAAS - DES.
- [] Existe un usuario dedicado de solo lectura para cada origen.
- [] El usuario tiene permiso SELECT sobre las tablas/vistas usadas por su script.
- [] Cada extracción dispone de una combinación de columnas estable y no nula para `MCAAS_KEY_COLUMNS`.
- [] Cada `EXTRACT_N_NAME` es único y estable.

4.2 Bases destino

- [] La tabla destino ya existe.
- [] Contiene todas las columnas devueltas por el SELECT y la columna indicada en `ORIGIN_FIELD_NAME`.
- [] La PK autoincremental del destino no comparte nombre con una columna que MCAAS deba insertar.
- [] El usuario tiene INSERT y UPDATE en destinos normales; INSERT en históricos.
- [] Firewall, puertos y TLS permiten acceso desde el servidor MCAAS.

4.3 Recomendación de índice

En destinos normales se recomienda una restricción o índice único sobre la identidad funcional. Ejemplo PostgreSQL:

```
CREATE UNIQUE INDEX ux_empleados_origen
ON empleados (source_name, empleado_id);
```

En destinos históricos use un índice normal, no UNIQUE, porque deben convivir varias versiones de la misma entidad.

4.4 Máquina de compilación y servidor

Solo la máquina que compila necesita Node.js y npm. El servidor Windows que recibe el ejecutable autocontenido no necesita Node.js. Durante la instalación se requieren permisos de Administrador para registrar las tareas programadas.

5. Estructura del proyecto

MCAAS-DES/	
assets/	Icono PNG e ICO
docs/	Manual
installer/	Script Inno Setup
scripts/	SQL, instalación, build y utilidades
src/	Código fuente TypeScript
db/	Adaptadores SQL Server/PostgreSQL/MySQL
tests/	Pruebas unitarias y smoke test
.env.example	Plantilla de configuración
package.json	Middle - Connector as a Service - Database Extractor n Sender
README.md	Resumen técnico
dist/windows/	Salida del build

6. Configuración del .env

En Windows, la configuración real queda por defecto en `C:\ProgramData\MCAAS-DES\.env`.

6.1 Múltiples extracciones

Variable	Descripción	Ejemplo
EXTRACTION_COUNT	Cantidad de trabajos de extracción definidos.	2
ORIGIN_FIELD_NAME	Nombre de la columna agregada por MCAAS a cada fila destino.	source_name
DESTINATION_AUTO_ID_COLUMN	PK autoincremental reservada del destino. MCAAS no la escribe.	id
EXTRACT_N_NAME	Nombre lógico único. Este valor se almacena en <code>ORIGIN_FIELD_NAME</code> .	Empleados_Matriz
EXTRACT_N_SCRIPT	Ruta al SQL de esa extracción.	./scripts/extraction-empleados.sql
EXTRACT_N_DB_TYPE	mssql, postgres o mysql.	mssql
EXTRACT_N_DB_URL	URL opcional de conexión.	vacío
EXTRACT_N_DB_HOST / PORT / NAME / USER / PASSWORD	Credenciales separadas si no usa URL.	127.0.0.1 / 1433 / CONTRPAQ
EXTRACT_N_TARGET_TABLE	Fallback opcional si el SQL no define <code>MCAAS_TARGET_TABLE</code> .	empleados
EXTRACT_N_SYNC_KEY_COLUMNS	Fallback opcional si el SQL no define <code>MCAAS_KEY_COLUMNS</code> .	empleado_id

```
EXTRACTION_COUNT=2
ORIGIN_FIELD_NAME=source_name
DESTINATION_AUTO_ID_COLUMN=id

EXTRACT_1_NAME=Empleados_Matriz
EXTRACT_1_SCRIPT=./scripts/extraction-empleados.sql
EXTRACT_1_DB_TYPE=mssql
EXTRACT_1_DB_HOST=127.0.0.1
EXTRACT_1_DB_NAME=CONTRPAQ_MATRIZ
EXTRACT_1_DB_USER=reader
EXTRACT_1_DB_PASSWORD=***

EXTRACT_2_NAME=Empleados_Sucursal_Norte
EXTRACT_2_SCRIPT=./scripts/extraction-empleados-sucursal.sql
EXTRACT_2_DB_TYPE=mssql
EXTRACT_2_DB_HOST=10.0.0.25
EXTRACT_2_DB_NAME=CONTRPAQ_NORTE
EXTRACT_2_DB_USER=reader
EXTRACT_2_DB_PASSWORD=***
```

Misma base, varias tablas: repita las mismas credenciales en dos `EXTRACT_N` y asigne un script distinto a cada trabajo.

6.2 Ciclo de sincronización

Variable	Uso	Default
POLL_INTERVAL_MS	Espera entre ciclos completos. Un ciclo recorre todas las extracciones.	2000
INTER_TARGET_DELAY_MS	Espera después de cada destino, haya éxito o fallo.	300
BATCH_SIZE	Filas máximas por transacción de destino.	250
HEARTBEAT_SECONDS	Frecuencia mínima de log cuando no hay cambios.	300

6.3 Destinos

`DESTINATION_COUNT` acepta valores de 1 a 10. Todos los datos de todas las extracciones se intentan enviar a todos los destinos.

Variable	Descripción
DEST_N_NAME	Nombre humano mostrado en logs.
DEST_N_DB_TYPE / DB_URL / HOST / PORT / NAME / USER / PASSWORD	Motor y credenciales del destino.
DEST_N_HISTORICO	<code>true</code> inserta una versión nueva; <code>false</code> mantiene una fila actual por identidad funcional.
DEST_N_TARGET_TABLE	Opcional. Si se usa, fuerza todas las extracciones de ese destino a una misma tabla.

6.4 Compatibilidad con v1.0.x

Si `EXTRACTION_COUNT` no existe, MCAAS aún acepta el formato anterior `SOURCE_DB_*` + `EXTRACTION_SCRIPT` como una sola extracción.

6.5 Estado, logs y fallos

Variable	Descripción	Default
STATE_DIR	Huellas de sincronización y estado persistente.	./state
LOG_DIR	Directorio de logs.	./logs
LOG_RETENTION_DAYS	Retención automática de logs diarios.	30
DATA_FLAG_PER_ROW	Registra hashes de cambio sin valores.	true
FAILURE_THRESHOLD	Número de fallos para considerar error persistente.	5
FAILURE_WINDOW_HOURS	Ventana temporal para contar los fallos.	1

7. Script SQL dinámico

Cada `EXTRACT_N` apunta a su propio script. El SQL define la tabla destino y las columnas clave de negocio.

```
-- MCAAS_TARGET_TABLE=empleados
-- MCAAS_KEY_COLUMNS=empleado_id

SELECT
    id AS empleado_id,
    nombre,
    puesto,
    fecha_modificacion
FROM dbo.Empleados;
```

Muy importante: si la tabla destino usa `id` como PK autoincremental y `DESTINATION_AUTO_ID_COLUMN=id`, no devuelva el ID de origen con nombre `id`. Use un alias como `empleado_id`, `cliente_id` o `source_id`.

Fila final enviada

Si `EXTRACT_1_NAME=Empleados_Matriz` y `ORIGIN_FIELD_NAME=source_name`, MCAAS transforma conceptualmente:

```
{ empleado_id: 25, nombre: "Ana", puesto: "Ventas" }

    en

{ empleado_id: 25,
  nombre: "Ana",
  puesto: "Ventas",
  source_name: "Empleados_Matriz" }
```

Los valores se envían como parámetros del driver, no concatenados directamente en SQL.

8. Detección de filas nuevas y actualizadas

La identidad funcional de una fila está formada por la columna de origen y las claves del script:

```
identidad = ORIGIN_FIELD_NAME + MCAAS_KEY_COLUMNS
```

Ejemplo:

```
(source_name="Empleados_Matriz", empleado_id=25)
(source_name="Empleados_Sucursal_Norte", empleado_id=25)
```

Aunque `empleado_id` sea igual, las identidades son diferentes.

Destino normal

1. MCAAS calcula un hash de la identidad y otro hash de toda la fila.
2. Si la huella de datos cambió, intenta UPDATE usando `source_name + empleado_id` (o las claves equivalentes).
3. Si el UPDATE no encuentra fila, ejecuta INSERT.

4. La columna autoincremental del destino, por ejemplo `id`, no aparece en el INSERT/UPDATE y la genera la BD.
5. Solo después del commit exitoso se actualiza el estado local del destino.

Esquema recomendado

```
CREATE TABLE empleados (  
  id BIGSERIAL PRIMARY KEY,  
  source_name TEXT NOT NULL,  
  empleado_id BIGINT NOT NULL,  
  nombre TEXT,  
  puesto TEXT,  
  UNIQUE (source_name, empleado_id)  
);
```

Resultado: el backend puede usar el `id` interno para relaciones locales y, al mismo tiempo, conocer exactamente de qué sistema salió la entidad mediante `source_name + empleado_id`.

9. Modo HISTORICO

Cuando `DEST_N_HISTORICO=true`, MCAAS conserva la detección de cambios por identidad funcional, pero no actualiza la fila existente. Cada versión nueva o modificada genera otro INSERT.

El `id` autoincremental del destino sigue siendo generado por la base, por lo que cada versión obtiene una PK distinta.

Ejemplo: (Empleados_Matriz, empleado_id=25) cambia de puesto. Un destino normal actualiza su fila; un destino histórico agrega una segunda versión.

No use `UNIQUE(source_name, empleado_id)` en una tabla histórica. Use un índice normal o una clave que incluya versión/fecha si necesita restricciones adicionales.

La semántica histórica continúa siendo at-least-once: una caída excepcional entre el commit remoto y el guardado del estado local puede repetir un INSERT.

10. Errores, reinicio y estado persistente

Fallo transitorio

Un error de conexión o escritura se registra y se vuelve a intentar. Los fallos de un destino no impiden que otros destinos ya procesados conserven su estado correcto.

Fallo fatal del ciclo

Si falla la extracción origen o el ciclo principal, el motor desconecta, espera `AUTO_RESTART_DELAY_MS` y vuelve a iniciar conexiones. Además, la tarea de Windows tiene política de reinicio si el proceso termina.

Error persistente

Si se alcanzan `FAILURE_THRESHOLD` fallos dentro de `FAILURE_WINDOW_HOURS`:

1. Se escribe `state/persistent-error.json`.
2. Se muestra el mensaje "Error persistente en curso; requiere atención".
3. Se intenta enviar un correo por Gmail API.
4. La aplicación permanece viva, pero pausa extracción y envío.
5. La pausa continúa aunque se reinicie Windows.

Cómo reanudar

Primero corrija la causa. Después ejecute:

```
mcaas-des.exe --clear-persistent-error --config "C:\ProgramData\MCAAS-DES\.env"
```

El servicio detectará la liberación en su siguiente verificación y reanudará la sincronización.

11. Logs y monitor

Archivo	Contenido
app-AAAA-MM-DD.log	Inicio, heartbeats, extracciones con cambios, envíos y mensajes comprensibles.
error-AAAA-MM-DD.log	Solo eventos de nivel ERROR.
data-flags-AAAA-MM-DD.ndjson	Banderas por cambio con hashes truncados, origen, destino y tipo NEW/UPDATED. No incluye valores de la fila.

El monitor abre una consola que hace tail del log de aplicación. Cerrar la consola no cierra el núcleo: el core corre como una tarea independiente.

```
mcaas-des.exe --monitor --config "C:\ProgramData\MCAAS-DES\.env"
```

Mensajes típicos:

```
[INFO] Produccion: 4 cambio(s) detectado(s) (3 nuevo(s), 1 actualizado(s)). Envío iniciado.  
[SUCCESS] Produccion: sincronización completada. Insertados: 3; actualizados: 1.  
[ERROR] Error persistente en curso; requiere atención. La sincronización quedó pausada.  
[INFO] Enviando correo de alerta mediante Gmail API...
```

12. Activación de Gmail API

MCAAS - DES usa OAuth 2.0 y el scope `https://www.googleapis.com/auth/gmail.send`. No necesita leer la bandeja.

12.1 Google Cloud

1. Cree o seleccione un proyecto en Google Cloud Console.
2. Abra APIs y servicios y habilite **Gmail API**.
3. En Google Auth Platform configure **Branding**, **Audience** y **Data Access**.
4. Si pertenece a Google Workspace y el proyecto será solo interno, use audiencia Internal cuando esté disponible. Si es External, agregue la cuenta emisora como usuario de prueba mientras desarrolla.
5. Cree un OAuth Client ID de tipo **Web application**.
6. Agregue como redirect URI autorizado: `http://localhost:53682/oauth2callback`.

12.2 Obtener refresh token

En una PC de desarrollo con Node instalado, copie `.env.example` a `.env` y cargue:

```
GMAIL_CLIENT_ID=...  
GMAIL_CLIENT_SECRET=...  
GMAIL_REDIRECT_URI=http://localhost:53682/oauth2callback
```

Luego ejecute:

```
node scripts/generate-gmail-token.mjs --config .env
```

Abra la URL mostrada, autorice la cuenta que enviará los avisos y copie el refresh token a:

```
GMAIL_ENABLED=true  
GMAIL_REFRESH_TOKEN=...  
GMAIL_FROM=alertas@tu-dominio.com  
GMAIL_ALERT_TO=soporte@tu-dominio.com
```

12.3 Producción

Importante: Google documenta que, para apps External en estado Testing, los refresh tokens pueden caducar después de 7 días. Para una operación 24/7, use un proyecto/estado de publicación apropiado para producción o una configuración Internal de Workspace cuando corresponda.

Referencias oficiales consultadas: Google Workspace Gmail API - Node.js quickstart, server-side authorization y sending messages; Google OAuth production readiness. Las políticas de OAuth pueden cambiar, por lo que deben revisarse antes de un despliegue productivo.

13. Pruebas y validación

13.1 Sin tocar bases

```
npm run typecheck  
npm test  
npm run validate
```

`validate` revisa el `.env`, todas las extracciones, todos los scripts, tablas objetivo y claves declaradas sin abrir conexiones.

13.2 Conectividad

```
npm run check:connections
```

Prueba **todos los orígenes** y todos los destinos. No sincroniza datos.

13.3 Una sola ejecución

```
npm run run:once
```

Ejecuta un ciclo completo: recorre todas las extracciones y las distribuye a todos los destinos.

13.4 Prueba específica de IDs repetidos

Antes de producción, prepare dos orígenes de staging donde exista el mismo ID de negocio y confirme que el destino crea dos filas diferenciadas por el campo de origen.

id	source_name	empleado_id
1	Empleados_Matriz	25
2	Empleados_Sucursal_Norte	25

13.5 Con el ejecutable compilado

```
mcaas-des.exe --validate-config --config C:\ruta\.env  
mcaas-des.exe --check-connections --config C:\ruta\.env  
mcaas-des.exe --run-once --config C:\ruta\.env  
mcaas-des.exe --status --config C:\ruta\.env
```

14. Build y empaquetado

14.1 Instalar dependencias

```
npm install
```

14.2 Build Windows autocontenido

```
npm run build:win
```

El comando ejecuta typecheck, pruebas, bundle con NCC, crea un ejecutable Windows x64 con `@yao-pkg/pkg` e intenta aplicar el icono/metadata MCAAS - DES con `resedit-cli`.

14.3 ZIP de release

```
npm run package:release
```

Genera una carpeta/ZIP para copiar a la PC servidor. El servidor no necesita Node.js.

14.4 Instalador Setup.exe opcional

En una máquina Windows con Inno Setup 6 instalado:

```
npm run installer:win
```

Salida esperada:

```
dist\windows\mcaas-des.exe
dist\windows\MCAAS-DES-Windows-x64\
dist\windows\MCAAS-DES-Windows-x64.zip
dist\windows\MCAAS-DES-Setup-1.1.0.exe (opcional)
```

15. Instalación en un servidor Windows de CONTPAQi

Opción A: carpeta de release + PowerShell

1. Compile MCAAS - DES en una PC de desarrollo.
2. Copie `MCAAS-DES-Windows-x64.zip` al servidor.
3. Extraiga la carpeta y abra PowerShell como Administrador.
4. Ejecute:

```
powershell -ExecutionPolicy Bypass -File .\install.ps1
```

El instalador copia el binario a `C:\Program Files\MCAAS-DES`, crea datos en `C:\ProgramData\MCAAS-DES` y copia todos los ejemplos `extraction*.sql` disponibles.

Configurar antes del arranque

1. Edite `C:\ProgramData\MCAAS-DES\.env`.
2. Defina `EXTRACTION_COUNT`, `ORIGIN_FIELD_NAME` y `DESTINATION_AUTO_ID_COLUMN`.
3. Configure cada `EXTRACT_N_NAME`, conexión y script.
4. Agregue/edite los SQL en `C:\ProgramData\MCAAS-DES\scripts\`.
5. Configure de 1 a 10 destinos.

Validar

```
"C:\Program Files\MCAAS-DES\mcaas-des.exe" --validate-config --config "C:\ProgramData\MCAAS-DES\.env"
"C:\Program Files\MCAAS-DES\mcaas-des.exe" --check-connections --config "C:\ProgramData\MCAAS-DES\.env"
"C:\Program Files\MCAAS-DES\mcaas-des.exe" --run-once --config "C:\ProgramData\MCAAS-DES\.env"
```

Activar 24/7

Ejecute `Start-MCAAS-DES.bat` como Administrador o inicie la tarea **MCAAS-DES Core** desde Task Scheduler. El monitor visible continúa separado del núcleo.

Opción B: Setup.exe

Si compiló Inno Setup, ejecute `MCAAS-DES-Setup-1.1.0.exe` como Administrador y realice la misma validación.

16. Operación diaria y recuperación

Comprobaciones periódicas

- [] El log diario contiene heartbeat reciente.
- [] Todas las extracciones esperadas aparecen en los logs.
- [] No existe error persistente activo.
- [] Los destinos reciben filas nuevas/modificadas con el origen correcto.
- [] La PK autoincremental del destino continúa siendo generada por la BD.

Agregar una extracción nueva

1. Aumente `EXTRACTION_COUNT`.
2. Agregue el bloque `EXTRACT_N_*` con un nombre único.
3. Cree el archivo SQL correspondiente.
4. Ejecute `--validate-config` y `--check-connections`.
5. Ejecute `--run-once` y verifique el destino.
6. Reinicie la tarea Core para cargar la nueva configuración.

Cambiar un `EXTRACT_N_NAME`

Trátelo como cambio de identidad. Al cambiar el nombre, las filas pasarán a tener un origen distinto y pueden insertarse como entidades nuevas. Mantenga el nombre estable salvo que ese comportamiento sea intencional.

Respaldos recomendados

- `.env`, protegido como secreto.
- Todos los `scripts/extraction*.sql`.
- `state/`, para conservar las huellas ya enviadas.

17. Seguridad recomendada

- Use un usuario de solo lectura en la base origen.
- Use usuarios de escritura limitados a las tablas necesarias en destinos.
- No reutilice cuentas administrativas de SQL Server/Aiven.
- Proteja `C:\ProgramData\MCAAS-DES\.env` con permisos NTFS restringidos.
- No incluya el `.env` real en Git, ZIP de soporte ni correo.
- Use TLS para bases remotas.
- Evite `SSL_REJECT_UNAUTHORIZED=false` en producción salvo que el proveedor/arquitectura lo requiera y el riesgo haya sido aceptado.
- Rote contraseñas y refresh tokens cuando cambie el personal responsable.
- Los logs de banderas no contienen valores, pero siguen siendo metadatos operativos; protéjalos.

18. Límites y supuestos

Tema	Comportamiento actual
Tiempo real	Polling configurable, no CDC/binlog nativo.
Deletes	No propaga eliminaciones del origen.
DDL	No crea ni migra tablas destino.
Esquemas	Cada target debe ser compatible con las columnas de su SELECT y con <code>ORIGIN_FIELD_NAME</code> .
IDs repetidos	Son válidos entre orígenes distintos; no son válidos dos veces dentro de la misma extracción para la misma combinación de claves.
PK autoincremental	Debe quedar fuera del SELECT usando alias para el ID de negocio cuando sea necesario.
Histórico	At-least-once; una caída en la ventana commit remoto/estado local puede producir duplicado.
Volumen	El estado local es JSON. Para datasets de millones de filas conviene evolucionar a SQLite/LevelDB, watermark o CDC.
Renombrar origen	Cambiar <code>EXTRACT_N_NAME</code> cambia la identidad funcional y puede provocar resinserción.

19. Referencia rápida de comandos

Desarrollo

```
npm install
npm run typecheck
npm test
npm run validate
npm run check:connections
npm run run:once
npm run dev
```

Build

```
npm run build:win
npm run package:release
npm run installer:win      # solo Windows + Inno Setup 6
npm run package:zip        # ZIP del proyecto fuente
```

Ejecutable

```
mcaas-des.exe --service --config C:\ruta\.env
mcaas-des.exe --run-once --config C:\ruta\.env
mcaas-des.exe --validate-config --config C:\ruta\.env
mcaas-des.exe --check-connections --config C:\ruta\.env
mcaas-des.exe --status --config C:\ruta\.env
mcaas-des.exe --monitor --config C:\ruta\.env
mcaas-des.exe --clear-persistent-error --config C:\ruta\.env
```

Checklist post-lanzamiento

- [] Cada `EXTRACT_N_NAME` es único y aparece correctamente en destino.
- [] Se probó al menos un ID repetido entre dos orígenes.
- [] El ID autoincremental del destino es generado por la BD, no por MCAAS.
- [] Los destinos contienen los registros esperados.
- [] La tarea MCAAS-DES Core está Running.
- [] Un reinicio de prueba confirmó autoarranque.